Commitment schemes

Pushpendra Pal

June 2025

1 What is a Commitment Protocol?

A commitment protocol is like a digital ”sealed envelope” - it allows someone to commit to a value while keeping it secret, then later reveal both the value and proof that it matches their original commitment.

1.0.1 Two Key Properties

- Hiding: The commitment reveals nothing about the committed value
- Binding: Once committed, the committer cannot change to a different value

Two Phases:

- Commit Phase: Create and publish commitment C = Commit(message, randomness)
- Reveal Phase: Show the message and randomness, others verify C matches

2 Specific Commitment Schemes

2.1 Pedersen Commitments

Setup: Choose group with generator g, another generator h where discrete log relationship is unknown
Commit: C = $g^m . h^r$

m = message to commit to
r = random blinding factor

Reveal: Send (m, r), verifier checks C = $g^m . h^r$

2.1.1 Properties

Computationally hiding (based on discrete log assumption)
Perfectly binding (cannot find two different openings)
Homomorphic: $C_1 . C_2 = g^{m_1+m_2} . h^{r_1+r_2}$

2.2 Schnorr-based Commitments

2.2.1 Method 1 - Direct Commitment

Commit: C = $g^m.h^r$ (same as Pedersen)
Uses Schnorr signature structure

2.2.2 Method 2 - Hash-based with Schnorr Proof

Commit: C = Hash(m, r)
Prove validity using Schnorr proof of knowledge
Reveal: Show (m, r) and verify hash or discrete log relation

2.3 Okamoto Commitments

Extension of Pedersen to multiple messages:
Setup: Generators g, h_1, h_2,..., h_n
Commit: C = $g^r.\ (h_1)^{m_1}.\ (h_2)^{m_2}.\ (h_n)^{m_n}$

Can commit to multiple values ($m_1, m_2, \ldots, m_n$) simultaneously r is randomness
Reveal: Send all ($m_1, m_2, ..., m_n$, r), verify the equation

2.3.1 Properties

- Multi-message capability
- Still computationally hiding and perfectly binding
- More flexible for complex applications